НРТК «ЛПР-1» — пояснения к коду

Состояние на 23.10.2026. Код в этой папке — **точная копия того, что работает на роботе**: программы Raspberry Pi сверены по SHA-256 с `~/robot/` на Pi, исходники прошивки платы — с рабочей папкой `D:\ai\telega\firmware`, готовая прошивка — та, что залита в плату (v5, SHA-256 `f1ee001d…`). Внутри каждого файла — подробные комментарии на русском (шапка файла: назначение, подключение, команды; у параметров — откуда взято число и когда замерено). Этот документ — путеводитель: как всё устроено целиком и что где искать.

Содержание папки

Папка	Что внутри
`raspberry - программы Pi/`	41 файл — всё содержимое `~/robot/` на Pi: программа управления, модули, диагностика, проверки без железа, служба `rc-drive.service`
`прошивка платы/исходники/`	исходники прошивки платы гироскутера (hoverboard-firmware-hack-FOC + наши правки), проект PlatformIO; `docs/pictures/mainboard_pinout.png` — распиновка платы
`прошивка платы/готовая/`	`firmware.bin` / `firmware.elf` — прошивка v5 (залита 14.10.2026); `original_factory_firmware.bin` — заводская прошивка платы (дамп 256 КБ, для возврата)
`инструменты ПК/`	скрипты Windows PowerShell: заливка программ на Pi, выгрузка журналов GPS, просмотр точек, добавление SSH-ключа, разбор журналов тока клюва

1. Как устроена система

```
Пульт FS-i6 ──2.4 ГГц──► приёмник FS-X6B ──i-BUS──► HW-597 (USB) ──► Raspberry Pi 3B+ ──UART 115200──► плата гироскутера ──► 2 мотор-колеса
                                                                                                          │
                                                                                                          ├─ I2C ─► PCA9685 №1 0x40 (посев): клюв 2 × MG996R 360°, челнок MG90S, переключатель камер W8010S
                                                                                                          │        INA219 0x41 / 0x44 — ток левого / правого серво клюва
                                                                                                          ├─ I2C ─► PCA9685 №2 0x42 (модуль руки): поворот, плечо, предплечье (MG996R 180°), захват (MG90S)
                                                                                                          ├─ USB ─► PL2303 → GPS NEO-M8N (журнал координат)
                                                                                                          ├─ USB ─► флешка VALLI (журналы GPS, ролики посева)
                                                                                                          └─ USB ─► камера Logitech C270 (ролик падения семени)
```

На Pi работает **одна программа** — `rc_drive.py`, её запускает служба `rc-drive` (systemd, пользовательская, автозапуск при включении). Она 50 раз в секунду читает каналы пульта и решает, что делать: ехать, сеять, двигать руку, какую камеру показать в очках. Посев, GPS и камера работают каждый в своём потоке и не мешают езде: если какого-то железа нет (не отвечает по I2C, нет USB-устройства), программа пишет об этом в журнал и работает без него.

Как связаны файлы

```
rc_drive.py ── главный цикл, логика пульта (класс Robot)
  ├── ibus.py        — чтение каналов пульта (i-BUS) в своём потоке
  ├── hoverboard.py  — обмен с платой гироскутера (команды, обратная связь) в своём потоке
  ├── hw.py          — драйверы I2C: PCA9685, INA219, общая шина с замком
  ├── camsel.py      — выбор камеры FPV (W8010S через PCA №1, кан. 8)
  ├── seeder.py      — посевной модуль: цикл посева в своём потоке (клюв, челнок, ток INA)
  │     └── shuttle.py — челнок бункера (используется и отдельно)
  ├── arm.py         — модуль руки: суставы и захват (PCA №2)
  ├── gps_log.py     — журнал GPS на флешку в своём потоке (трек + точка на каждый посев)
  └── seed_cam.py    — камера посева: кольцо видео в памяти, ролик на каждый сброс семени
```

Безопасность — где она в коде

Защита	Где
Колёса — только при ARM (SwA вниз) и после того, как стики побывали в центре	`rc_drive.py`, `Robot.step()`
Нет кадров пульта дольше 0.3 с → стоп, камера 1, идущий посев прерывается, клюв поднимается	`Robot.step(None)`, `RC_TIMEOUT`
Программа зависла → драйвер сам шлёт «стоп» через 0.3 с; программа упала → плата сама останавливается через 0.8 с	`hoverboard.py` (`command_timeout`), прошивка `main.c`
Во время посева и при «клине» клюва колёса стоят	`Seeder.blocks_drive()`
Посев и рука — никогда одновременно (общий источник 6 В)	`Robot.step()` / `_sow()` / `_manip()`
Перед движением клюва: на обеих INA ≥ 5.0 В и ток покоя ≤ 0.5 А, иначе ошибка и клюв не двигается	`Seeder._power_ok()`
INA не видят ток серво (провод мимо шунта) → стоп с ошибкой	`seeder.py`, `NO_CURRENT_A`
Перезапуск службы (`systemctl stop/restart`) → `finally`: клюв на «стоп», порты закрыты	`rc_drive.py`, `main()` + `rc-drive.service` (`KillSignal=SIGINT`)
Ошибка I2C не роняет программу — модуль выключается с сообщением	`Robot._io()`

2. Программы Raspberry Pi (`raspberry - программы Pi/`)

2.1 Основная программа (работает в службе)

`rc_drive.py` — управление с пульта

- **Главный цикл** (`main()`): открывает I2C и платы PCA, запускает GPS, камеру, посев, руку, читатель пульта и драйвер платы; дальше каждые 20 мс — `robot.step(каналы, dt)`. Раз в 5 с (и при смене состояния) пишет строку в журнал: состояние, пределы скорости, обороты гусениц, напряжение батареи, температура платы, состояние посева, руки, камеры, GPS.
- **Класс `Robot`** — вся логика пульта, железо подаётся снаружи (поэтому её можно проверить на ПК с подставным железом). `step()` на каждый кадр каналов: ARM, режим «ход / манипулятор» (SwD), камера, посев по SwC, движение. `_sow()` — запуск одного посева на переключение SwC в «вниз» (только при ARM, стоящей платформе, стиках в центре, в режиме «ход»). `_manip()` — режим руки: стики = скорость суставов, газ = захват.
- **Настройки в начале файла:** каналы пульта (`SPEED_CH`, `STEER_CH`, `ARM_CH`, `MODE_CH`, `SEED_SW_CH`, `GRIP_CH`), пределы скорости (`KNOB_MIN/MAX_SPEED` 40…200 об/мин, `KNOB_MIN/MAX_STEER` 60…160), мёртвая зона стиков 8 %, инверсии (`INVERT_STEER = True` — под прошивку v5), какие модули включены (`SEEDER`, `ARM_MODULE`, `CAMSEL`, `SEED_CAM`, `GPS_LOG`), оси руки (`ARM_AXES`: сустав → канал пульта и инверсия).
- Функции `stick()` (1000…2000 → −1…1 с мёртвой зоной) и `pos3()` (трёхпозиционный тумблер → 0 / 1 / 2).

`ibus.py` — пульт (i-BUS) Класс `IBusReader`: в своём потоке читает кадры i-BUS (32 байта, 14 каналов, контрольная сумма) из HW-597 по постоянному имени `/dev/serial/by-id/usb-1a86_USB_Serial-if00-port0`. Если адаптер пропал (переподключение USB) — переоткрывает порт каждые 0.5 с; при старте ждёт порт, а не падает. `channels(max_age)` — последние каналы или `None`, если кадров давно не было. Сам по себе — монитор каналов: `python3 ibus.py`.

`hoverboard.py` — плата гироскутера Класс `Hoverboard`: протокол прошивки FOC по USART (`/dev/serial0`, 115200). Команда — 8 байт `0xABCD, steer, speed, checksum`; поток шлёт её каждые 20 мс (плата требует пакет не реже 0.8 с). Обратная связь — 18 байт (обороты моторов, напряжение батареи, температура платы); поток ищет начало кадра в потоке байт. `drive(speed, steer)`, `stop()`, `fresh_feedback()`. `command_timeout=0.3`: если программа перестала обновлять команду — драйвер сам шлёт стоп.

`hw.py` — железо I2C

- `LockedBus` — шина SMBus с замком: посев работает в своём потоке, без замка два потока перепутали бы адреса.
- `Pca9685` — плата серво: 50 Гц по **замеренной** частоте генератора (у клонов не 25 МГц: №1 — 25.84 МГц, №2 — 25.185 МГц); `set_us(канал, мкс)`, `off(канал)` (серво отпущено), `get_us(канал)` (что выдаёт сейчас — по нему рука подхватывает суставы без рывка).
- `Ina219` — датчик тока: шунт 0.1 Ом, **1 замер без усреднения** (у клонов аппаратное усреднение врёт, сглаживает программа).
- Адреса: 0x40 PCA посева, 0x42 PCA руки, 0x41 / 0x44 INA левая / правая.

`camsel.py` — выбор камеры FPV Класс `CamSelect`: импульс на кан. 8 PCA №1 → вход S переключателя W8010S. `PULSES = {1: 2000 (ходовая, IN1), 2: 1000 (рука, IN3), 3: 1500 (посев, IN2)}` — проверено в очках. Импульс держит PCA сама, пишется только при смене камеры. Команды: `status`, `1/2/3`, `us N`, `scan` (перебор импульсов — найти, какая камера на каком).

`seeder.py` — посевной модуль «семена 112» Самый большой модуль. Клюв ходит вверх-вниз на рейках от двух серво **360°** (импульс = скорость, не угол), поэтому положение клюва программа **считает по времени** (`FAST_MM_S` 55 мм/с вниз, `UP_MM_S` 60 вверх) и **ловит упоры по току** двух INA219 (концевиков нет).

Цикл посева (`_cycle()`, ≈ 5 с, клюв в грунте ≈ 3.5 с):

Упор при подъёме раньше 60 % спуска → «КЛИН» (камень, корень): езда запрещена до сброса (SwC вверх → середина или ARM выкл → вкл).

1. если клюв не вверху (после запуска, ошибки) — сначала поиск верха;
2. клюв вниз до касания по току (рост суммы тока против неё же 0.3…0.1 с назад) или до предела «до земли + глубина» = `GROUND_MM` 30 + `DEPTH_MM` 15 мм от верха (в рыхлом грунте ток растёт поздно — обычно срабатывает предел); торможение командой «стоп» 0.3 с (без сигнала левое серво докручивается);
3. пауза 1 с → челнок «набор» 1000 мкс → «сброс» 1400 мкс → отметка ролика камеры и точка GPS;
4. клюв вверх: медленно ~12 мм, пауза 0.3 с (створки раскрываются над лункой), быстро, у верха — до упора по току или по перескоку зуба шестерни по рейке (всплеск тока), запасной стоп — по расчётному положению + 0.06 с.

Главные параметры (в начале файла, у каждого — когда и как замерен): `N_L / N_R` 1510 — «стоп» серво; `FAST_D` 270 — ход импульса от «стопа»; `DOWN = +1` — какая сторона «вниз» (перепроверять после переборки шестерён); `TRIM_CCW_US` 15; `FILL_US` 1000 / `DROP_US` 1400 / `HOME_US` 1400 — челнок; пороги упора `STALL_*`, касания `DOWN_*`, перескока `SKIP_*`; `RACK_MAX_MM` 63 — дальше рейки не пускают; `TOP_MARGIN_S` 0.06. **Настройки без правки кода** — файл `~/robot/seeder_settings.json`, команда `seeder.py set` (`set ground 30`, `set depth 15`, `set dip 10`, `set ground default`); служба читает их перед каждым посевом. **Память положения** — `~/.seeder_pos.json`: если клюв был вверху, после

перезапуска поиск верха не нужен (клюв не дёргается). **Ручные команды** (службу остановить): `status`, `read 5`, `idle`, `home`, `up`, `jog 0.3`, `soft 240`, `cycle`, `cycle --top`, `cycle 5`, `set …` — каждая движущая пишет ток в `~/robot/ina_logs/beak_….csv` (разбор на ПК — `инструменты ПК/beak_replay.py`).

`shuttle.py` — челнок бункера Класс `Shuttle`: MG90S 180° на кан. 2 PCA №1; плавный ход 800 мкс/с; набор 1000, сброс и «дом» 1400. В покое канал **не выключается**: PCA держит импульс сама, и после пропадания 6 В серво возвращается на место. Команды: `status`, `home`, `fill`, `drop`, `cycle N`.

`arm.py` — модуль руки Класс `Arm`: суставы `JOINTS` (канал, имя, мин, «старт», макс, скорость мкс/с) и захват `GRIP` (канал, открыт, закрыт, скорость). `activate()` — принять управление: сустав, у которого PCA уже держит импульс, подхватывается с места; выключенный (после включения питания) — встаёт в «старт» рывком. `update(dt, скорости, захват, k)` — стик = скорость сустава в пределах `JOINTS`; захват плавно идёт за положением газа. `Arm(grip=None)` — захват не трогать. Калибровка 23.10.2026: поворот 550…2450 (старт 1150), плечо 1580…2400 (1600), предплечье 600…2100 (1500), захват 900 / 1900, 200 мкс/с. Команды: `status`, `set ИМЯ МКС [--wide] [--speed=N]`, `park`, `off [ИМЯ]`.

`gps_log.py` — журнал GPS Класс `GpsLogger` в своём потоке: читает NMEA 9600 (`$..GGA` — координаты, решение, спутники, HDOP; `$..RMC` — дата, скорость, курс) с PL2303. Пишет **только на флешку VALLI** (`/media/biba/VALLI/valli_logs/NNNN_<дата>/`): `trek.csv` — трек 1 Гц, `posadka.csv` — точка на каждый посев (`mark()`: на стоянке — среднее за 3 с, на ходу — интерполяция между фиксами), `nmea.log` — сырые строки. Номер посадки — сквозной по всем сессиям. Поправка «лунка относительно антенны»: `HOLE_FWD_M` 0.05 м вперёд, `HOLE_RIGHT_M` −0.15 (15 см левее) — применяется, если известен курс. Нет флешки — строки копятся в памяти и дописываются, когда она появится.

`seed_cam.py` — камера посева Класс `SeedCam`: ffmpeg непрерывно пишет камеру C270 (640×480, 30 к/с, MJPEG как есть, выдержка 1 мс) кусками по 1 с в кольцо в памяти `/dev/shm/seedcam` (12 кусков). На каждый сброс семени (`event()`) склеивает куски −1…+1.5 с вокруг сброса в ролик на флешку (`…/seedcam/posev_ЧЧММСС_NNN.mkv`), имя ролика — в заметке точки посадки. `analyze()` — заготовка распознавания «семя упало» (нужен OpenCV, пока не установлен).

`rc-drive.service` — служба Пользовательская служба systemd: `~/.config/systemd/user/rc-drive.service`, рабочая папка `~/robot`, журнал раз в 5 с (`RC_LOG_PERIOD=5`), перезапуск при падении через 2 с, остановка сигналом SIGINT (программа успевает остановить моторы и поставить клюв на «стоп»). Команды: `systemctl --user status|stop|start|restart rc-drive`, журнал — `journalctl _SYSTEMD_USER_UNIT=rc-drive.service -f`.

2.2 Проверки без железа

Файл	Что проверяет
`test_modules.py`	35 проверок: подставные PCA9685 / INA219 на подставной шине, **модель клюва** (рейки, верхний упор, перескок зуба, грунт, клин), подставные пульт, плата, рука, камеры. Логика `Robot`: ARM, режимы, посев, рука без/с захватом, потеря сигнала. Запускается автоматически при каждой заливке — не прошла → на Pi ничего не меняется
`test_gps_log.py`	26 проверок `gps_log.py` и `gps_export.py`: подставные NMEA, часы, «флешка» во временной папке; сквозной номер, интерполяция, поправка антенны, выгрузка GPX / KML / GeoJSON

Запуск: `python3 test_modules.py` (на Pi или на ПК с Python 3.10+). В конце — «ВСЁ ВЕРНО» или список ошибок.

2.3 Выгрузка и просмотр GPS

Файл	Назначение
`gps_export.py`	журналы сессий → `posadka.gpx` / `.kml` / `.geojson` (точки + трек) и общий `vse_posadki.*` — на той же флешке

`gps_points.py`	только чтение: последние посадки, трек, состояние GPS; `-f` — следить, `--all` — сводка по сессиям

2.4 Диагностика и стенды (запускаются вручную)

Файл	Назначение
`full_diag.sh`	полная диагностика Pi только чтением: питание (throttled), температура, память, диски, ошибки ядра, Wi-Fi, USB, UART / I2C, PCA9685, служба, sha256 программ
`diag.sh`, `power_watch.sh`, `usb_ports.sh`, `usb_watch.sh`	питание и USB; `power_watch.sh 30` — просадки каждые 3 с
`pca_regs.py`	обе PCA9685 только чтением: режим, частота, что выдаёт каждый канал (можно при работающей службе)
`pca_diag.py`, `pca_measure.py`	регистры PCA / держать импульс для мультиметра; замер настоящего импульса и частоты генератора (провод с канала → GPIO27)
`i2c_probe.py`	шина I2C программно, без включения I2C и без sudo
`ina_raw.py`, `ina_test.py`	сырые регистры INA219; стенд тока серво с записью в CSV
`servo_test.py`, `servo_deadband.py`, `servo_check.sh`	проверка одного серво (GPIO или PCA, плавно); поиск «стопа» серво 360°; что готово на Pi
`cam_check.sh`	камера посева без остановки службы: режимы, кольцо, кадр (`auto` — кадр с автовыдержкой)
`gps_cat.py`, `gps_gpio.py`, `uart_loop.py`	сырые строки NMEA; GPS прямо на GPIO27 без адаптера; петля TX ↔ RX адаптера
`deploy_remote.sh`	вторая половина заливки на Pi: проверка → бэкап → замена → перезапуск службы
`check_channels.sh`, `dir_check.sh`, `failsafe_test.py`, `listen.py`, `rc_ranges.py`, `spin_test.py`	ранние проверки пульта и платы (каналы, направления, failsafe, вращение)
`seed_drum.py`	**не используется** — барабан прежнего посевного модуля (14.10.2026), оставлен для истории

3. Прошивка платы гироскутера (`прошивка платы/`)

Основа — открытый проект hoverboard-firmware-hack-FOC (EFeru), сборка PlatformIO, окружение `VARIANT_USART`. Чип платы — GD32F103RCT6 (совместим с STM32F103RCT6). **Наши правки:**

Файл	Что изменено
`Inc/config.h`	`VARIANT_USART`: управление и обратная связь по **USART2** (левый боковой разъём, PA2 / PA3), 115200; `CTRL_MOD_REQ = SPD_MODE` — команда = обороты колеса (об/мин), `N_MOT_MAX` 1000, `I_MOT_MAX` 15 А, `I_DC_MAX` 17 А; `INVERT_R_DIRECTION` + `INVERT_L_DIRECTION` (v5: «вперёд» платы = вперёд робота); `INACTIVITY_TIMEOUT 0` (не выключаться по простою); `TEMP_WARNING_ENABLE 1` (5 сигналов при 60 °C); батарея 10S, калибровка напряжения `BAT_CALIB_*`
`Src/main.c`	аварийная остановка при потере связи 0.8 с и запрет моторов, пока связи нет; проверка простоя пропускается при `INACTIVITY_TIMEOUT 0`; писк `BEEPS_BACKWARD` — при движении **назад**

`Src/util.c`	устойчивый разбор потока USART2 (поиск начала кадра `CD AB`, исправлено зависание при `pos == len` буфера DMA)
`platformio.ini`	окружение `VARIANT_USART`

Сборка: `%USERPROFILE%\.platformio\penv\Scripts\pio.exe run -d <папка исходники> -e VARIANT_USART` → `.pio/build/VARIANT_USART/firmware.bin`. Пересборка из этих исходников побайтно совпадает с `готовая/firmware.bin`.

Заливка (ST-Link V2: SWDIO → PA13, SWCLK → PA14, GND; 3.3 В ST-Link не подключать; плата — от батареи, кнопку питания держать всю заливку; скорость 240 кГц — на скорости по умолчанию заливка однажды оборвалась):

```
openocd -f interface/stlink.cfg -c "transport select swd" -f target/stm32f1x.cfg -c "adapter speed
240" -c "init" -c "reset halt" -c "stm32f1x mass_erase 0" -c "program firmware.elf verify reset exit"
```

(полная команда с путями — `CLAUDE.md`, «Железо и доступы»). Вернуть заводскую прошивку — так же, `original_factory_firmware.bin` (адрес 0x08000000).

Программа Pi и прошивка — только вместе: v5 работает с `rc_drive.py`, где `INVERT_SPEED = False`, `INVERT_STEER = True`.

4. Скрипты ПК (`инструменты ПК/`)

Скрипт	Назначение
`deploy_pi.ps1`	**заливка программ на Pi одной командой**: `deploy_pi.ps1 -Files a.py,b.py [-NoRestart]` — проверка маршрута к Pi, копирование, на Pi — `deploy_remote.sh` (тесты, бэкап в `~/robot/backup/<дата>/`, замена, перезапуск службы). Пути внутри — под папку проекта `D:\ai\telega`
`gps_download.ps1`	выгрузка журналов GPS с флешки в Pi на ПК (`gps_export.py` на Pi + копирование)
`gps_points.ps1`	просмотр точек на флешке с ПК (`-N 30`, `-Follow`, `-All`)
`add-ssh-key.ps1`	добавить SSH-ключ ПК на Pi (вход без пароля); пароль Pi — у владельца
`beak_replay.py`	прогон журналов тока клюва (`~/robot/ina_logs/*.csv`) через детектор `seeder.py` — подбор порогов без движения клюва

Все `.ps1` с кириллицей сохранены в UTF-8 с BOM (иначе PowerShell 5.1 читает их неверно). Пульт FS-i6 при работе по SSH выключать — глушит Wi-Fi Pi.

5. Где менять, если…

Нужно	Где
другая скорость / пределы езды	`rc_drive.py`: `KNOB_*`, `MAX_*` (сначала — тест на земле)
стик руки едет не в ту сторону	`rc_drive.py`: `ARM_AXES` (инверсия)
пределы / «старт» суставов руки, захват	`arm.py`: `JOINTS`, `GRIP`
модуль посева поднят / опущен, другая глубина	без правки кода: `python3 seeder.py set ground MM`, `set depth MM`
клюв после переборки шестерён едет не туда	`seeder.py`: `DOWN`
другая камера на другом входе W8010S	`camsel.py`: `PULSES`
смещение лунки от антенны GPS	`gps_log.py`: `HOLE_FWD_M`, `HOLE_RIGHT_M`

выключить модуль целиком	`rc_drive.py`: `SEEDER`, `ARM_MODULE`, `CAMSEL`, `SEED_CAM`, `GPS_LOG` = `False`

После любой правки — `python3 test_modules.py` (и `test_gps_log.py` для GPS) на ПК, потом `deploy_pi.ps1 -Files …`.